

Lab 3: Likelihood!

NRES 746

Fall 2025

Maximum likelihood and optimization

These next couple weeks are focused on estimating model parameters and confidence intervals using maximum likelihood methods.

Likelihood represents the probability of drawing your particular data set given a fully specified model (e.g., a regression model with a particular vector of coefficients, θ):

$$L(\theta) = P(data|\theta)$$

This lab is designed to take two lab sessions to complete.

As with all labs in this course, your answers will either take the form of R functions (submitted as part of an R script) or short written responses submitted via WebCampus.

Please complete all assignments by the due date listed in WebCampus. You can work in groups but please submit the materials individually.

When writing functions, it's often easiest to start off by writing a script, and then when you have everything working properly, wrap the script into a function.

Don't forget to test your functions!

And remember that your final submission should only contain your functions- not any ancillary code you may have used to develop and test your functions.

Please comment your code thoroughly. Don't be afraid of *for* loops and inefficient coding practices, as long as it gets the job done!

Example: reed frog predation data

Load the reed frog predation data from the Bolker book- it can be found here or loaded from the 'emdbook' package ("ReedfrogPred"). Save this file to your working directory.

This dataset represents predation data for *Hyperolius spinigularis* (Vonesh and Bolker 2005). You can read more about this data set in the Bolker book.

```
# Reed frog example -----  
  
rfp <- emdbook::ReedfrogPred # load the data using the 'emdbook' package:  
head(rfp)
```

Because predation on tadpoles is size and density-dependent, we will subset these data to a single size class ('small') and density (10) for all treatments including a predator (this simplifies the problem!). Subset your data now:

```
# Take a subset of the data
```

```
rfp_sub <- subset(rfp, (rfp$pred=='pred') & (rfp$size=="small") & (rfp$density==10))
rfp_sub
```

For each individual, the per-trial probability of being eaten by a predator is a binomial process (i.e., they can survive or die during the interval). Recall that the likelihood that k out of N individuals are eaten as a function of the per capita predation probability p is:

$$Prob(k|p, N) = \binom{N}{k} p^k (1-p)^{N-k}$$

Since the observations are independent, the joint likelihood of the whole data set is the product of the likelihood of each individual observation. So, if we have n observations, each with the same total number of tadpoles N , and the number of tadpoles killed in the i th observation is k_i , then the likelihood is:

$$L = \prod_{i=1}^n \binom{N}{k_i} p^{k_i} (1-p)^{N-k_i}$$

Here we assume the data are binomially distributed – the binomial distribution is the natural choice for data that are represented as k ‘successes’ out of N ‘trials’. We conventionally work in terms of the log-likelihood (l), which is:

$$l = \sum_{i=1}^n \left[\log \binom{N}{k_i} + k_i \log(p) + (N - k_i) \log(1-p) \right]$$

In R we can represent the data likelihood as follows:

```
rfp_sub$killed <- with(rfp_sub, density-surv) # make column for number killed in each replicate trial
with(rfp_sub, sum(dbinom(killed, 10, prob=0.5, log=TRUE)) ) # expression of data likelihood(log scal
```

```
## [1] -12.8038
```

There is only one parameter (“unknown”) in this calculation: p .

We know how many individuals we started with ($N = 10$ for each trial) and how many were killed/eaten in each trial ($k = 3, 5, 1$, and 1). So we want to solve for the most likely value of p given our observations of N and the number of tadpoles eaten.

In essence we do this by picking a possible value of p (which can only range from 0 to 1), calculating the log-likelihood using the equation above, picking another value of p , completing the equation, etc. until we exhaust all possible values of p and identify the one having the highest log-likelihood value.

Of course R has useful built in functions to help us optimize the likelihood function!

The “`dbinom()`” function calculates the likelihood for a binomially distributed response variable.

Here, the response variable is the number of tadpoles eaten, and we are modeling the probability p of being eaten by a predator!

Given our observed k (number killed), and $N = 10$ for each trial, what is the likelihood of our data given that that p is 0.5?

```
L = dbinom(rfp_sub$killed, size=10, prob=0.5) # evaluate data likelihood with p=0.5
L
```

```
## [1] 0.117187500 0.246093750 0.009765625 0.009765625
```

What is the likelihood of observing all 4 of our outcomes, assuming independent replicates?

```
prod(L)      # joint data likelihood
```

```
## [1] 2.750312e-06
```

The joint likelihood value gets smaller and smaller each time we add more data (can you see why?), and can become very tiny very quickly. This is one of the reasons we prefer to work with log-likelihoods (which yield larger numbers and have very nice mathematical properties).

Taking the log of a value between 0 and 1 (like a probability mass) yields a negative number, which is why we often see that our log likelihood values are negative.

We can build on these ideas to estimate the likelihood over the entire possible parameter space (probability of being eaten- which can range from 0 to 1).

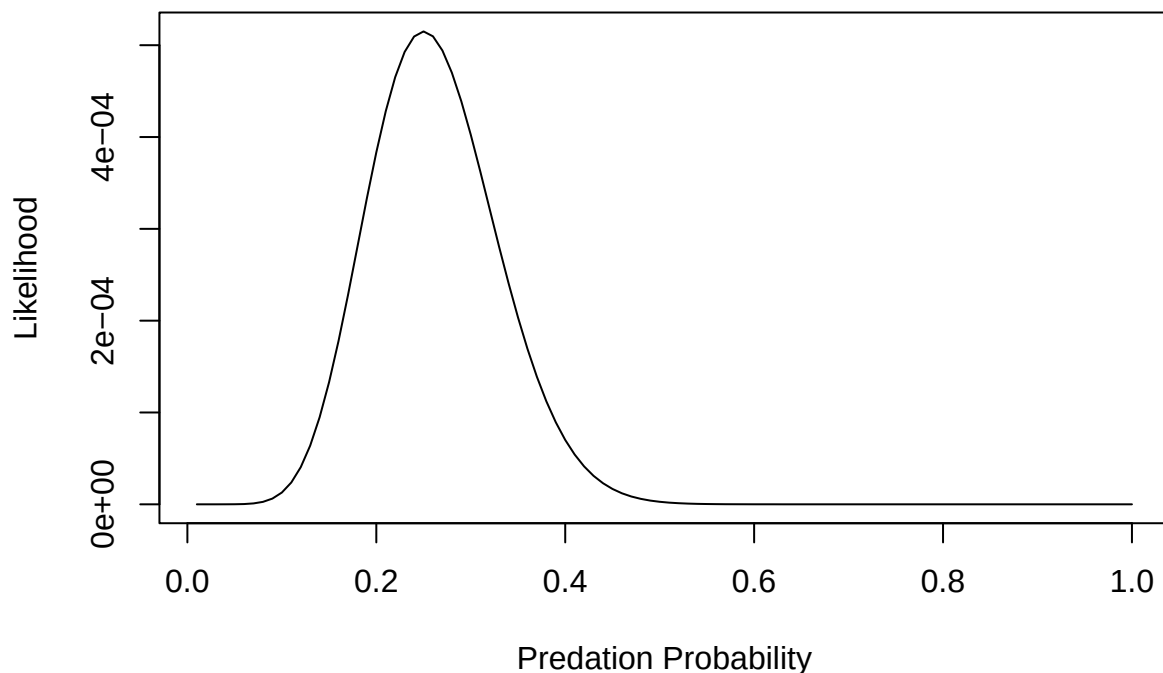
First we generate a sequence of plausible values for our free parameter, p_{eaten} .

```
p_seq <- seq(0.01, 1, length=100)      # prepare for visualizing the likelihood across parameter space
```

For every value of p we will calculate the binomial probability of the data.

```
Lik <- sapply(p_seq, function(t) prod(dbinom(rfp_sub$skilled,10,prob=t)) )
```

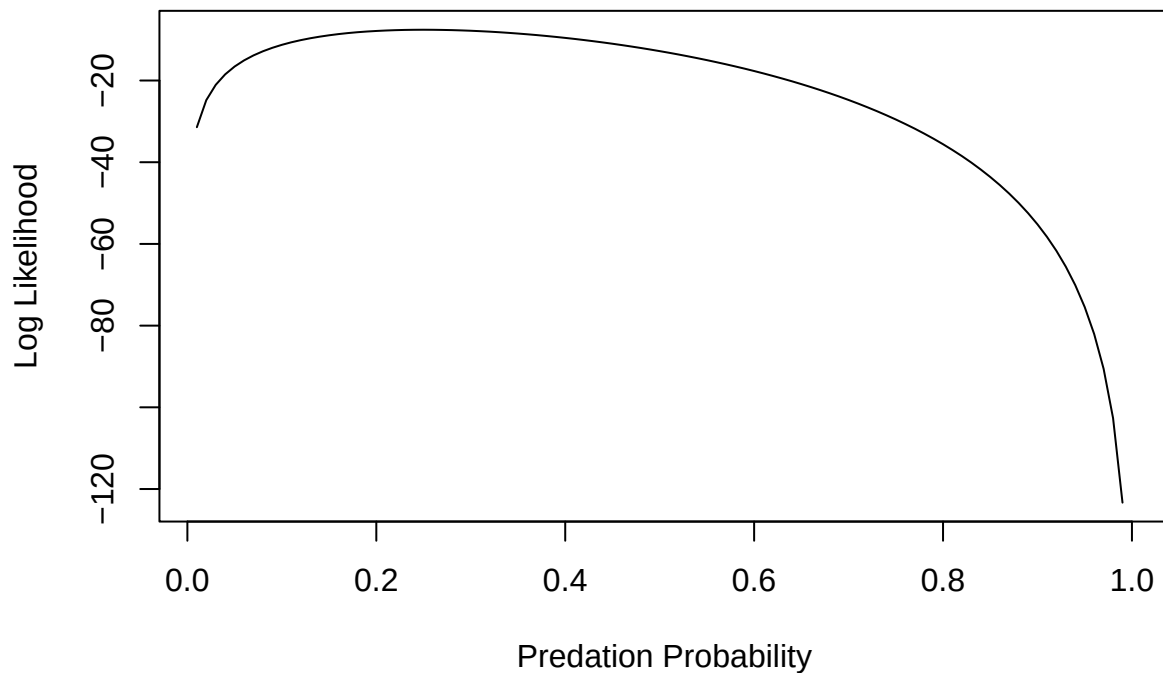
```
plot(Lik~p_seq,lty="solid",type="l", xlab="Predation Probability", ylab="Likelihood")
```



Let's look at the log-likelihood:

```
# plot out the log-likelihood
```

```
LogLik <- sapply(p_seq, function(t) sum(dbinom(rfp_sub$killed,10,prob=t,log=T)) )
plot(LogLik~p_seq,lty="solid",type="l", xlab="Predation Probability", ylab="Log Likelihood")
```



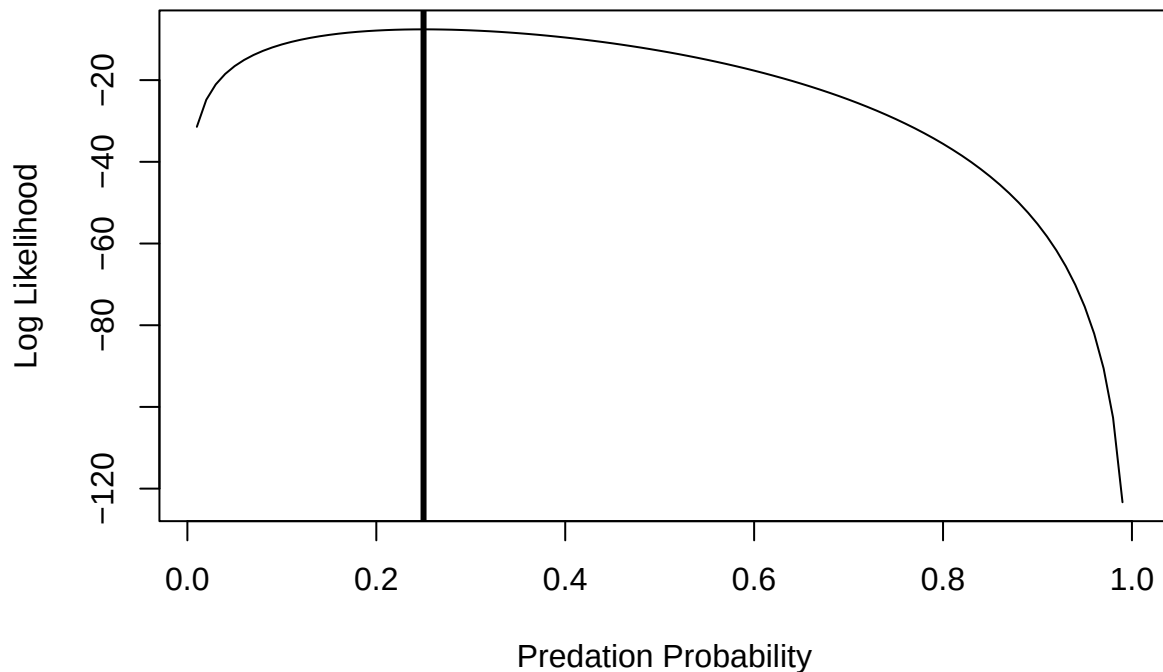
The **maximum likelihood estimate (MLE)** is the parameter value that maximizes the likelihood function.

```
p_seq[which.max(LogLik)]      # MLE for probability of predation
```

```
## [1] 0.25
```

And we can add an “abline()” to indicate the maximum Log-Likelihood estimate:

```
plot(LogLik~p_seq,lty="solid",type="l", xlab="Predation Probability", ylab="Log Likelihood")
abline(v=0.25,lwd=3)
```



Alternatively, we can use the `optim()` or `mle2()` functions to find the maximum likelihood estimate.

Although we want the maximum likelihood estimate, in practice we generally minimize the negative log-likelihood. To do this, we first write a function to calculate the negative log-likelihood function:

```
# Write a likelihood function

binomNLL1 <- function(p) {
  -sum(dbinom(rfp_sub$skilled, size=10, prob=p, log=TRUE))
}
```

As we did in class, you can use the `optim()` function to minimize your negative log-likelihood function (`'binomNLL1()'`) given a vector of starting parameters and your data. The starting parameters need not be accurate, but do need to be reasonable for the function to work, that's why we spent time in class eyeballing curves (also read the Bolker book for a discussion of the 'method of moments', which can help you get reasonable starting values!). Given that there is only one estimable parameter, p , in the binomial function, you only need to provide a starting estimate for it. Calculate the negative log-likelihood:

```
# use "optim()" to find the MLE

opt1 <- optim(fn=binomNLL1, par = c(p=0.5), method = "BFGS") # use "optim()" to estimate the parameter
```

You may get several warning messages, can you think why? `opt1` returns a list that stores information about your optimization process.

```
opt1 # check out the results of "optim()"
```

```
## $par
##      p
```

```
## 0.2500002
##
## $value
## [1] 7.571315
##
## $counts
## function gradient
##      17      7
##
## $convergence
## [1] 0
##
## $message
## NULL

MLE = opt1$par
MaxLik = opt1$value
```

The important bits are the parameter estimate and the maximum likelihood value.

But the first thing to check is whether or not the optimizer achieved convergence.

```
opt1$convergence
```

```
## [1] 0
```

Here a value of 0 means convergence has been achieved, a value of 1 means the process failed to converge. There is more info about convergence and alternative optimization options in Chapter 7 of the Bolker book.

This numerically computed answer is (almost exactly) equal to the theoretical answer of 0.25.

Plot your observed outcomes against your predictions under the maximum likelihood model:

```
hist(rfp_sub$skilled,xlim=c(0,10),freq=F,main="",xlab="Number of tadpoles killed")
curve(dbinom(x,prob=MLE,size=10),add=T,from=0,to=10,n=11,lwd=2, col="darkgreen")
```



Note that “freq=F” scales the y-axis of a histogram to “density”, which allows us to overlay probability density functions.

Exercise 3.1: building a custom likelihood function

In this exercise you will develop and use a *likelihood function* for the following scenario: you are a frog biologist studying a rare species of a wetland-obligate frog species. You visit three known-occupied wetland sites ten (10) times and for each site you record the number of visits for which your target species is detected (at least one call within a 5 minute period). Your data are as follows: 3,2 and 6 detections for each of the three sites respectively.

Exercise 3.1a.

Write a likelihood function called “NLL_frogOccupancy()” for computing the negative log-likelihood for the above scenario. Your likelihood function should compute the negative log-likelihood of these data: [3,2 and 6 detections for sites 1, 2, and 3 respectively] for any detection probability p , assuming that all sites are continuously occupied. You should assume for now that all sites have the same detection probability.

- **input:**
 - p = a single number specifying a proposed value (initial value) for the parameter “ p ” (probability of detection for a single visit – which happens to be the only free parameter in this model)
- **suggested algorithm:**
 - up to you! (HINT: use binomial distribution) (NOTE: do not use ‘optim()’ here: you are just asked to compute the likelihood at any specified parameter (input argument “ p ”), not to find the maximum likelihood estimate)
- **return:**
 - the negative log likelihood of your data (a single number)

Include your function in your submitted r script!

And test your function! For example:

```
NLL_frogOccupancy(p=0.5)    # test your function
```

```
## [1] 6.853154
```

Exercise 3.1b.

On WebCampus please respond briefly to the following questions:

1. What is the approximate maximum likelihood estimate for the p (detection probability) parameter? How did you use your likelihood function to get your answer (briefly)?
2. Using the “rule of 2”, what is the approximate 95% confidence interval for the p (probability of detection) parameter.

Exercise 3.2: Adding a Deterministic Relationship

So we’ve looked at how to obtain the likelihood given a stochastic model (the binomial distribution), but now we can consider more interesting questions – like when the effects of covariates on our process of interest.

Recall that we subset our data above because we expected survival to be (in part) density-dependent. Here we’ll consider how to model the probability of tadpoles being eaten as a function of the initial density of tadpoles in the population. Specifically, the response represents the number of tadpoles eaten by the predator (dragonfly larvae) over the 2-week experiment. The predictor variable is the initial density of tadpoles in each mesocosm.

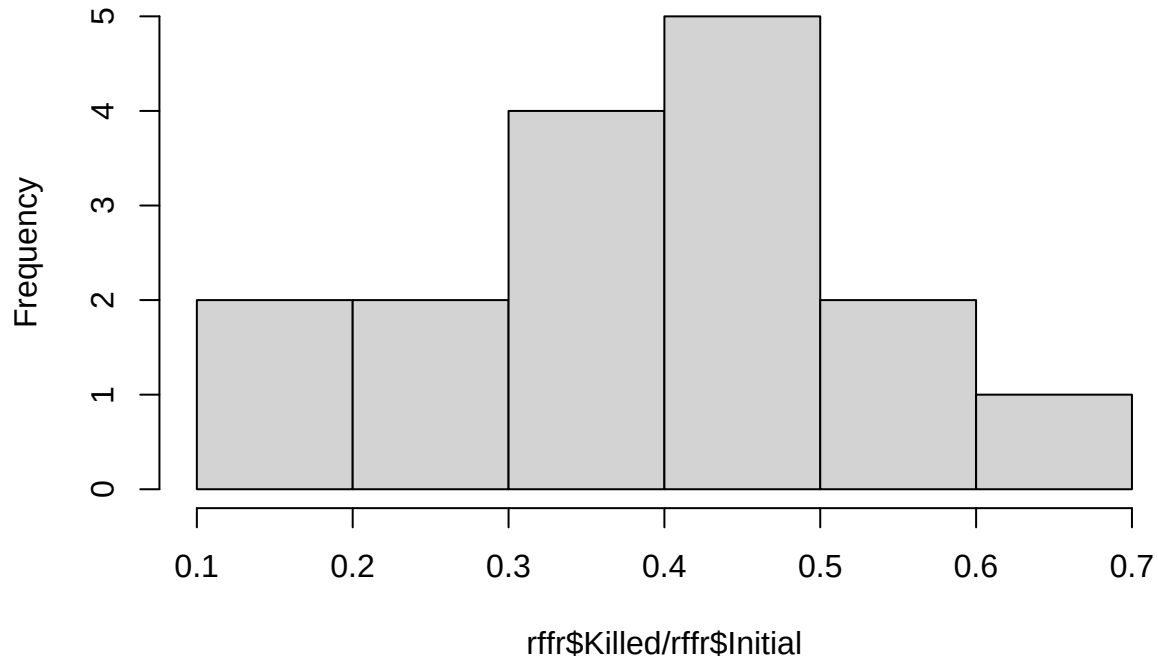
First, examine the first few lines:

```
# 3.2a -----  
  
rffr <- emdbook::ReedfrogFuncresp      # from Bolker's "emdbook" package  
# ?Reedfrog      # learn more about this dataset  
head(rffr)
```

Let’s look at the distribution of the data (probability of being killed).

```
hist(rffr$Killed/rffr$Initial)
```


Histogram of `rffr$Killed/rffr$Initial`



Based on what we know about the data, let's use a binomial distribution to describe the observed number of tadpoles killed.

Plot the number of tadpoles killed as a function of the initial density (e.g., using `ggplot()`) to see what sort of deterministic function would describe the pattern.

It looks like it could be linear, but we know that predators become handling-limited (saturated) at high prey densities.

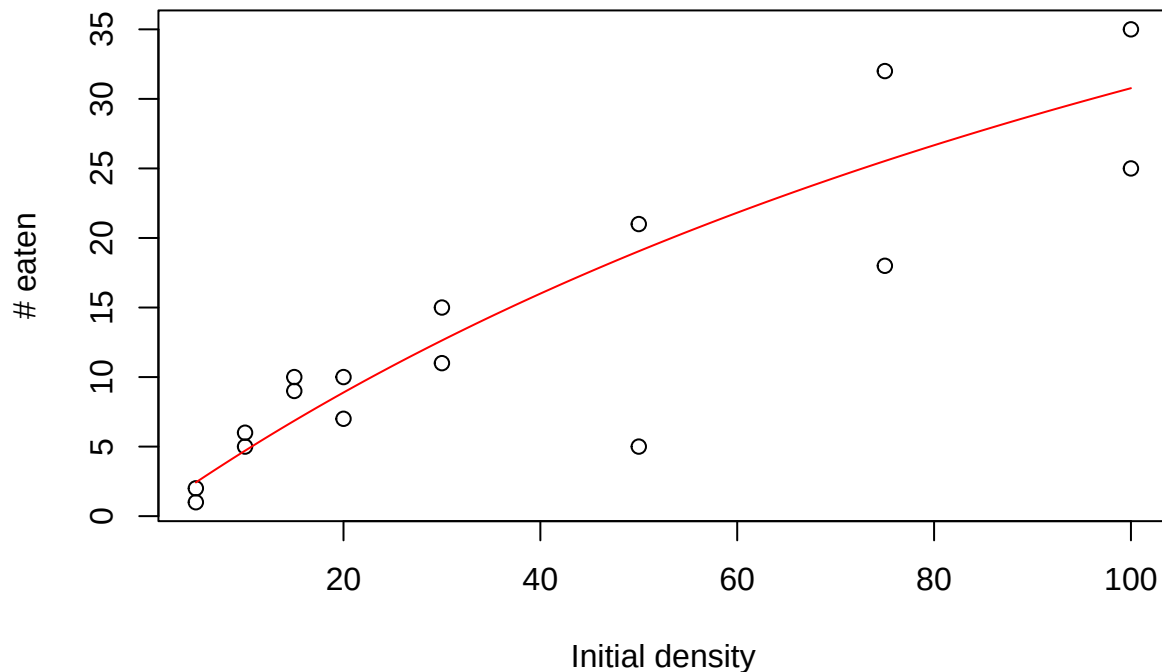
On page 182 Bolker indicates that if predation rate = $aN/(1 + ahN)$ (saturation curve, known in ecology as a Holling Type II functional response), this means that the per-capita predation rate of tadpoles decreases with tadpole density hyperbolically:

$$(\text{per-capita predation rate}) = a/(1 + ahN)$$

We'll use this deterministic function for our data.

First, let's see what that saturation curve would look like over our data points with an initial guess at the parameters (we always need an initial guess to get our optimization algorithm started).

Recall that the a parameter of this hyperbolic function indicates the initial slope, which we'll guess to be around 0.5, and the h parameter indicates $1/\text{asymptote}$, which we fiddled around with to match the data (so try $1/80$).



This looks pretty good, but we want to actually fit the line to the data instead of making guesses, and we'll use maximum likelihood to do that.

Just like before, we'll write a *negative log likelihood function*, but this time we'll incorporate the deterministic model!

Exercise 3.2a.

Write a function called “binomNLL2()” for computing the data likelihood for this model. This likelihood function should reflect that the data are drawn from a binomial distribution (either killed or not), and that the probability of predation (the number killed divided by the initial number) is explained by the Holling type II equation.

- **input:**
 - params = named vector of initial values for the params to estimate (length 2, with elements named ‘a’ and ‘h’ from the Holling type II functional response- in that order)
- **suggested algorithm:**
 - up to you! Make sure you use the Holling type II function to model the number killed at each density. Use the ‘dbinom()’ function to represent the stochastic process that generated these data. Note that the probability of being eaten (p parameter in the binomial distribution) can be computed as the expected number killed divided by the total initial density.
- **return:**
 - the negative log-likelihood of the data under this model

You can test your function using something like the following:

```
binomNLL2(c(a=0.4,h=1/200),c(5,10,15),c(3,5,6))
```

```
## [1] 4.706114
```

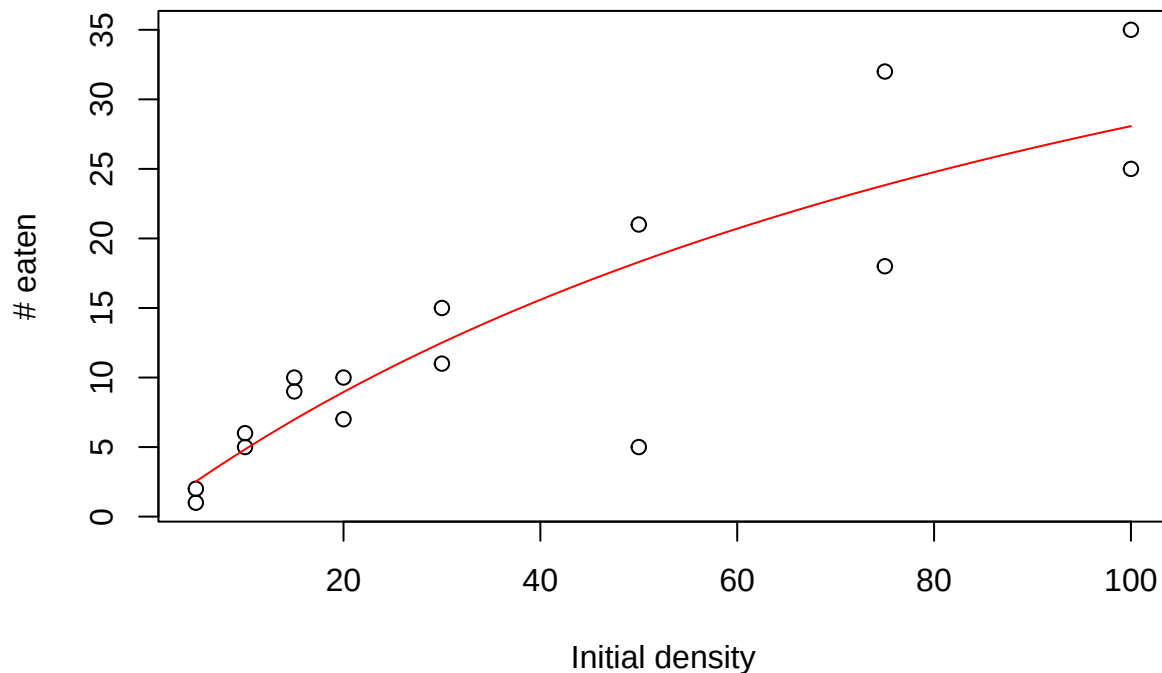
Now we can find the parameter values that best describe these data using ‘optim()’. We’ll use the same initial values for a and h that we used to plot the curve. N is the initial number of tadpoles, and k is the number of tadpoles killed.

```
opt2 <- optim(c(a=0.5,h=(1/80)), binomNLL2, N=rffr$Initial, k=rffr$Killed) #use default simplex algorithm
MLE = opt2$par
MaxLik = opt2$value
```

The results are not that different from our starting values, so we made a good guess!

Let’s visualize the results!

```
plot(rffr$Killed~rffr$Initial, xlab="Initial density",ylab="# eaten")
curve(Holl12(x, a=MLE["a"], h=MLE["h"]), add=TRUE,col="red")
```



Exercise 3.2b.

Write a function called “Rffuncresp()” for plotting the goodness-of-fit for this model.

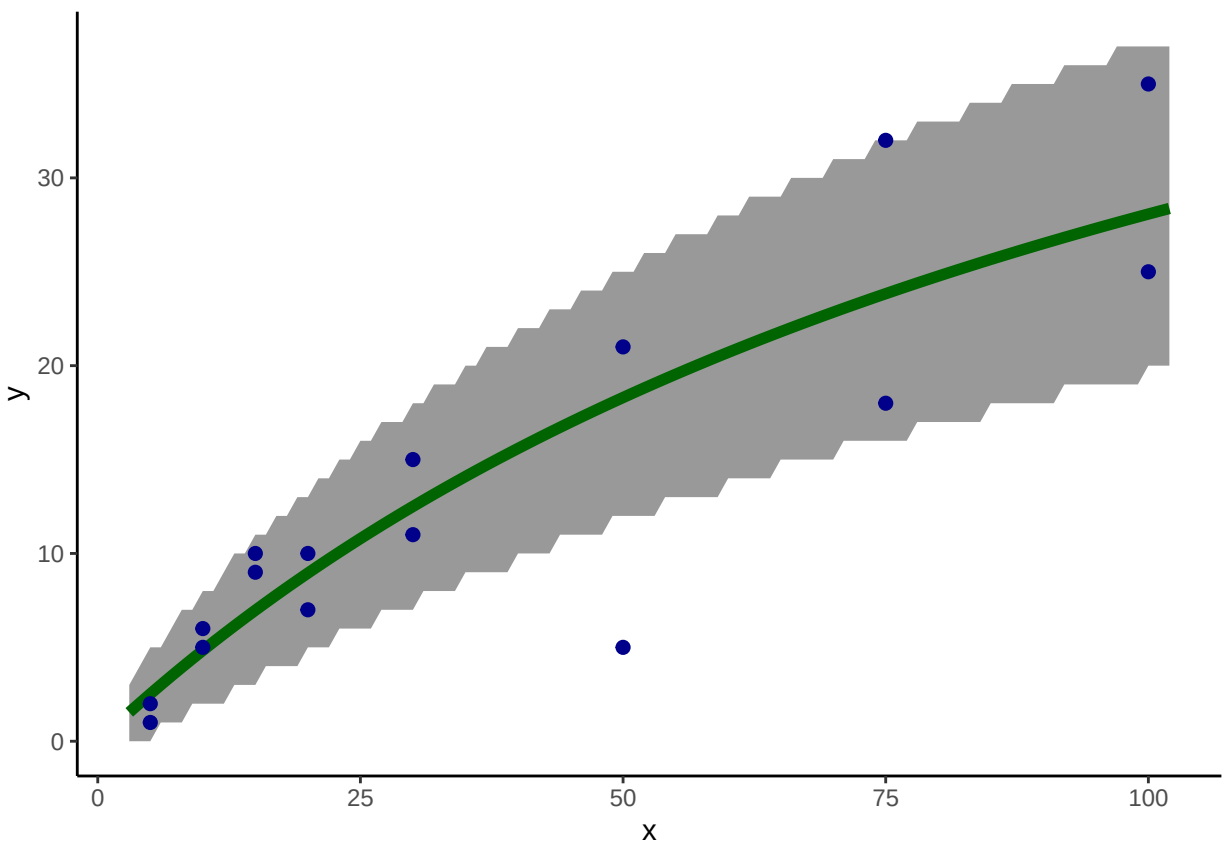
- **input:**
 - params = named vector containing the MLE for the params in this model (length 2: a and h from the Holling type II functional response- in that order, named “a” and “h”, respectively)
 - data = a data frame of 2 columns and one row per observation. The first column should represent the initial prey densities, and the second column should represent the number eaten by the predator over the 2-week experiment.
- **suggested algorithm:**
 - Plot the observed number killed (y axis) vs the initial densities

- Overlay a line to visualize the expected number killed based on the MLE parameters.
- Finally, visualize “plug-in prediction intervals” around your MLE line to make a plot like Figure 6.5a in the Bolker book. You can use “qbinom()” to define the 90% quantiles of the binomial distribution for every point along your curve (see below).
- **return:**
 - a data frame with 4 columns: The first column should represent a vector of initial densities that span the range of values in the experiment the second column should represent the expected number killed based on the MLE, the third and fourth columns should represent the lower and upper bounds of the 90% ‘plug-in’ confidence interval, respectively.

Include your function in your submitted r script!

You can test the function using something like this:

```
Rffuncresp(params=MLE,dat=rffr)
```



NOTE: the “prediction interval” you are asked to generate here is sometimes called a “*plug-in*” prediction interval, and is a quick and dirty way to assess model goodness-of-fit. You just take the best-fit model (with parameter values at their MLE values) and use the quantiles of the stochastic process (here, the binomial distribution) to define the range of data that would generally be produced under this model.

NOTE: For this exercise, you can use the ratio of the tadpoles killed versus the initial tadpole density values to get a probability (probability of being killed), which you can then use as an input parameter for the binomial distribution. For example:

```
# how to generate 'plug-in' prediction intervals
```

```
xvec <- seq(40,80,5)
yvec <- 0.5/(1+0.5*0.015*xvec) * xvec
```

```
upper<-qbinom(0.975,prob=yvec/xvec, size=xvec)
lower<-qbinom(0.025,prob=yvec/xvec, size=xvec)

upper
lower
```

Exercise 3.2c.

On WebCampus, respond briefly to the following question:

Bolker calls the prediction interval you generated above a *plug-in prediction interval*. In what way(s) is this interval different than a *true* prediction interval? Would you feel comfortable reporting a ‘plug-in’ prediction interval in a scientific publication, or to stakeholders interested in your model results?

Review of the MLE process

Step 1. Identify the response and explanatory variables (e.g., Predation probability and Initial Population Size). Just stating what the response and explanatory variables are will help you start modeling.

Step 2. Determine one or more candidate stochastic process(es) that may govern variability among observations in your system (e.g., error is Binomially distributed). In the tadpole case, the stochastic distribution was easy to identify because the data were in the form of successes in a finite set of trials (which yields a binomial distribution under independence assumptions). Other times it may not be so clear what the best distribution is, and looking at the histogram and plotting different distributions over the top will be helpful.

Step 3. Specify the deterministic function – e.g., a mathematical function that governs the relationship between a predictor variable and the mean response (e.g., Holling type II). Here, we chose this function mechanistically – we expect predator functional response to saturate with increasing prey density.

Step 4. Write a function for expressing the negative log-likelihood of the data given our deterministic expectations and the stochastic distribution. Our negative *log likelihood function* in the tadpole functional response example combined the stochastic and deterministic elements together by having the stochastic parameter (in this case the binomial probability, p) be dependent upon the deterministic parameters.

Step 5. Make a guess for the initial parameters (e.g., $a=0.5$, $h=1/80$). You need to have an initial guess at the parameters to make ‘optim()’ work, and we plotted the Holling curve to make our guess. Sometimes you will also need to make a guess at the parameters for the stochastic distribution. In these cases, the *method of moments* is often a good and simple option (see Bolker book for details).

Step 6. Estimate the best-fit parameters using maximum likelihood. We used optim() to search through all possible value combinations of parameters a and h to estimates for those parameters that correspond to the minimum negative log-likelihood.

Step 7. Add confidence or prediction intervals around your estimates to represent uncertainty. We calculated some *plug-in estimates* to put pseudo-prediction intervals around our estimates based on the stochastic distribution.

Exercise 3.3: Likelihood and Confidence Intervals

Let’s continue the myxomatosis virus titer example from the optimization lecture. The difference is that this time we’ll model a deterministic process (change in viral loads over time) in addition to the stochastic process. Our goal is to fit a model to data on viral titers through time for the viruses that are grade 1 (highest virulence). Biological common sense, and data from other titer levels, tells us to expect titer levels to start at zero, increase over time to a peak, and then to decline. Given those expectations, we’ll fit a Ricker model to these data, following Bolker’s example and extending it just a bit.

Our goals are to:

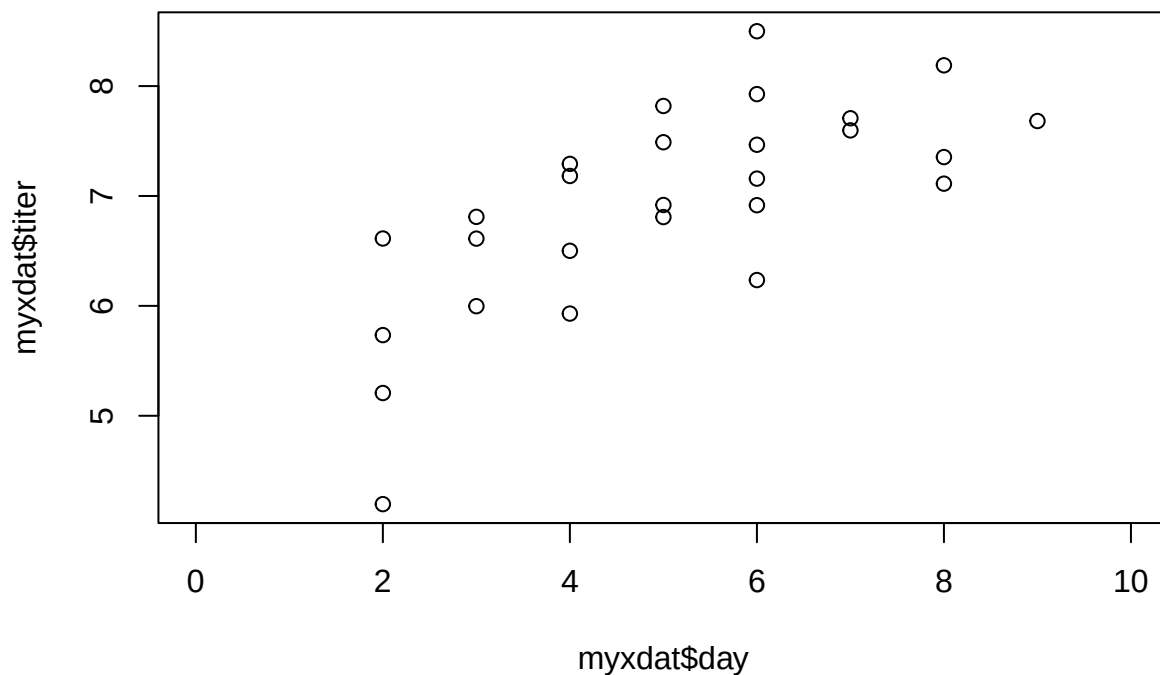
- Find the maximum likelihood estimates of the parameters of the Ricker model fit to the myxomatosis data.
- Visualize the fit of this model to the data by:
 - Plotting the data
 - Adding the predicted Ricker curve
 - Adding plug-in confidence intervals
 - Computing confidence intervals for MLE params
 - Computing confidence intervals for predictions

You can add the data from the ‘emdbook’ package. Select just the grade 1 titers:

```
# Exercise 3.3a -----

library(emdbook)
data(MyxoTiter_sum)      # load the data
# head(MyxoTiter_sum)
myxdat <- subset(MyxoTiter_sum, grade==1)  # select just the most virulent strain

plot(myxdat$titer~myxdat$day,xlim=c(0,10))  # visualize the relationship
```



Exercise 3.3a.

Fit a Ricker model to the myxomatosis data – that is, use a Ricker deterministic equation to model the expected (mean) virus titer as a function of days since infection (see below for step-by step instructions).

Write a function called “NLL_myxRicker()” for computing the data likelihood and plotting the goodness-of-fit for this model.

- **input:**
 - params = named vector of initial values for the params to estimate (length 3: *a* and *b* params from the Ricker model [see below], and the *rate* parameter of the Gamma distribution)(names should be “a”, “b”, and “rate”)
- **suggested algorithm:**
 - Compute the deterministic function: use a Ricker deterministic equation to model the mean virus titer as a function of days since infection
 - Compute the “shape” parameter of the Gamma distribution as a function of (1) mean virus titer and (2) the “rate” parameter of the Gamma distribution [NOTE: the variance and the mean of the Gamma distribution are dependent- so we can’t simply model the expected value and the noise separately! See below for more details]
 - Compute the sum of the negative log-likelihoods of all observations (use the “dgamma()” function)
 - NOTE: you can assume the existence of the “myxdat” data object in the workspace, exactly as in the above code block.
- **return:**
 - the negative of the sum of the log-likelihoods of all observations (a single number)

This task (develop likelihood function) can be broken down into a few steps, just like we did above!

Our question is: how does a virus titer change in rabbits as a function of time since infection? This is almost exactly the same problem as we just did (above), but we’re using different distributions and functions.

Step 1. *Identify the response and explanatory variables.* The response is the virus titer and the explanatory variable is the days since infection.

Step 2. *Determine the stochastic distribution.*

Start by plotting the histogram of the response variable. Bolker suggests a gamma distribution – does it look like a gamma would work? Write down the parameters for the gamma distribution (page 133). (For the gamma distribution, use with the shape and scale parameters (not the rate)).

Step 3. *Specify the deterministic function.*

Plot the data points (hint: look at figure 6.5b). Bolker suggests the Ricker curve – does it look like the Ricker curve would work? Write down the equation and parameters for the Ricker curve (page 94).

Step 4. *Specify the likelihood of the data given our deterministic expectations and the stochastic distribution.*

Take a moment to think how the parameters of the stochastic distribution are determined by the parameters of the deterministic function. For the gamma distribution, both the shape and scale parameters are related to the mean of the distribution, i.e., $\text{mean} = \text{shape} / \text{rate}$ (page 133)(or $\text{shape} * \text{scale}$). So how will you specify that the deterministic function (the Ricker model) should represent the mean? What parameters do you need your (negative) log-likelihood function to estimate? Write out your negative log-likelihood function to solve for the likelihood.

ASIDE: the mean and variance of the gamma distribution are inter-related: the shape parameter can be specified as: $\frac{\text{mean}^2}{\text{var}}$ and the rate parameter can be specified as $\frac{\text{mean}}{\text{var}}$!

Include your function in your submitted r script!

You can test your function using something like this:

```
NLL_myxRicker(params=c(a=4,b=0.2,rate=2))    # test the function
```

```
## [1] 44.56117
```

We can then optimize our likelihood function using ‘optim()’. This takes us through the next couple steps in the maximum likelihood process outlined above.

Step 5. *Make a guess for the initial parameters.* We need initial parameters to put into “optim()”. Remember that the Ricker curve parameters can be estimated based on the initial slope and maximum (see pg. 95). Try plotting the curve over the points to get an approximate fit. There really isn’t any easier way to get there than trial and error for the deterministic function.

Step 6. *Estimate the best fit parameters using maximum likelihood.*

Now use ‘optim()’ to get your maximum likelihood parameter estimates.

```
temp <- optim(NLL_myxRicker,par = c(a=2,b=0.2,rate=2))
MLE = temp$par
MinNLL = temp$value
```

Exercise 3.3b.

Write a function called “predict_myxRicker1()” for computing the maximum likelihood estimates and plotting the goodness-of-fit for this model.

- **input:**
 - mle = named vector of MLE values for the model params (length 3: *a* and *b* params from the Ricker model [see below], and the *rate* parameter of the Gamma distribution) (named “a”, “b”, and “rate”, in that order)
- **suggested algorithm:**
 - make a data frame with a column “x” spanning the range of the time-since-infection (predictor variable, in days). [I suggest making this vector at least 100 elements]
 - add a column with the predictions from the Ricker model based on the MLE parameters (just the Ricker parameters) across the range of times since exposure.
 - add columns to denote the lower and upper bounds of the plug-in prediction interval around your MLE predictions. Hint: use “qgamma()” to define the 95% quantiles of the Gamma distribution for every point along your curve (see below).
 - Use ggplot to plot the observed virus titer (y axis) vs the days since infection (x axis).
 - Overlay a line to visualize your predictions from the Ricker model
 - Finally, visualize “plug-in prediction intervals” around your MLE line to make a plot similar to Figure 6.5a in the Bolker book.
- **return:**
 - df: a data frame with 4 columns:
 - * x, representing a vector ranging across the time span of the experiment (in units of days)
 - * mean, representing the expected virus titer under the MLE parameters
 - * lwr, representing the lower bound of the plug-in prediction interval
 - * upr, representing the upper bound of the plug-in prediction interval

NOTE: you can assume (1) that a function “NLL_myxRicker()” is available in the global environment and that the “myxdat” data object is also available for use.

To complete this exercise will involve going through the final step (7) in the process outlined above:

Step 7. *Add confidence/prediction intervals around your estimates.*

After your run of ‘optim()’ (did you achieve convergence?), plot your fitted Ricker curve to your data. Revisit the earlier prediction interval code to add *plug-in* prediction intervals around your predicted curve based on gamma distributed errors (should resemble Figure 6.5b on page 184 of text).

NOTE: You can use the “qgamma()” function to build your plug-in intervals!

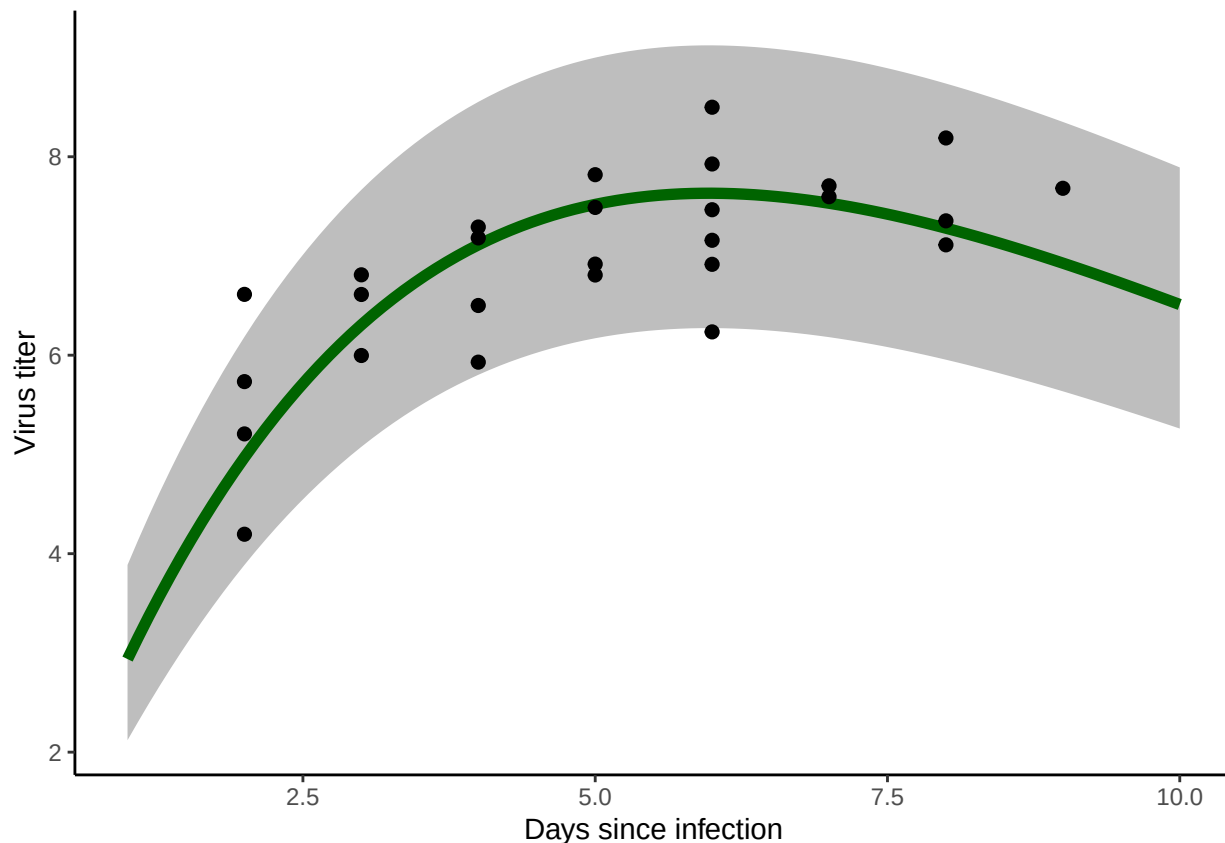
```
# Plug-in prediction intervals!
```

```
upper<-qgamma(0.975,shape=?, rate=?) # remember that the mean of the gamma distribution is shape*scale
lower<-qgamma(0.025,shape=?, rate=?)
```

Include your function in your submitted r script!

And as always, make sure to test your function code using something like this:

```
predict_myxRicker1(mle=MLE) # test the function
```

Exercise 4: Confidence Intervals on parameters

In this exercise you will construct confidence intervals that reflect our uncertainty about the parameter estimates in the Ricker/virus titer example. We will then construct prediction intervals that include our uncertainty about the parameter values.

Again, you can assume that there is a data frame in your environment named ‘myxdat’ with columns named ‘days’ and ‘titer’ that contain the data.

Exercise 4.1a: Gaussian approximation

The first step is to optimize your likelihood function, this time using the “hessian=T” argument to the ‘optim’ function:

```
temp <- optim(NLL_myxRicker, par = c(a=2, b=0.2, rate=2), hessian = T)
MLE = temp$par
MinNLL = temp$value
H = temp$hessian
H
```

```
##           a           b           rate
## a      224.5874208 -4080.39276  0.27045577
## b     -4080.3927576 83586.61145 -4.73994026
## rate    0.2704558  -4.73994  0.06511682
```

To compute confidence intervals, we first need to compute the standard error of the parameters. This is pretty easy if we remember that the MLE is asymptotically multivariate normally distributed with a variance-covariance matrix of H^{-1} , where H is the hessian matrix (matrix of second partial derivatives of the

negative log-likelihood function with respect to each parameter) evaluated at the MLE. In other words, the variance-covariance matrix of the parameter vector at the MLE is approximately (asymptotically) equal to the inverse of the hessian matrix of the negative log-likelihood function, evaluated at the MLE. This sounds complicated but is pretty simple to do in R.

Once you have computed the variance covariance matrix for the MLE parameters, you can compute the standard error as the square root of the diagonals of the variance covariance matrix (the diagonal elements represent variances). Finally, to compute a margin of error for a confidence interval, you can multiply the standard error by the appropriate quantile from a standard normal distribution- for example, for a 95% confidence interval you would multiply the sterr by 1.96, which is the 0.025 quantile of the standard normal distribution (if you exclude 0.025 from the top and the bottom of the distribution, the middle portion contains 95% of the probability mass. Finally, you can add and subtract this margin of error to compute a 95% confidence interval around each parameter.

Write a function called “CI_myxRicker1()” for computing a confidence interval for all free parameters in this model.

- **input:**
 - mle = named vector containing the maximum likelihood values for the params (length 3: *a* and *b* params from the Ricker model [see below], and the *rate* parameter of the Gamma distribution)(names should be “a”, “b”, and “rate”)
 - H = the hessian matrix obtained from optimizing the negative-log-likelihood function (i.e., from ‘optim()’ with ‘hessian=T’)
- **suggested algorithm:**
 - compute variance-covariance matrix (VCM) from the hessian matrix as described above.
 - extract the diagonal elements of the VCM and take the square root- this is the standard error of your parameters.
 - compute the margin of error by multiplying the standard error of each parameter by the appropriate quantile of the z distribution (standard normal). In this case you want the 97.5% quantile.
- **return:**
 - a matrix: each row is a parameter (in order: a, b, and rate) and there are four columns: first, the mle values, second, the standard error, third, the lower bound of the confidence interval, and fourth, the upper bound of the confidence interval.

HINT: to take the inverse of a matrix in R, you can use the “solve()” function.

You can test your function code using something like this:

```
CI_myxRicker1(MLE,H)    # test the function

##           mle           se          lwr          upr
## a      3.4736659 0.19852047 3.084573  3.8627589
## b      0.1674329 0.01028582 0.147273  0.1875927
## rate 14.4231248 3.92873690 6.722942 22.1233077
```

Exercise 4.1b: confidence intervals on predictions

Write a function called “predict_myxRicker2()” that computes a 95% confidence interval for predicted titer values across a range of days-since-exposure.

To do this, we will use a nice trick called the **delta method** for computing the standard error of the expected titer computed from the MLE. The delta method is applicable any time we want to compute the standard error for a transformation of an asymptotically normally (or multivariate-normally) distributed quantity/vector. Mathematically, it is based on a Taylor expansion, so it is important that the transformation function is differentiable.

In this class, we will use the delta method to find the variance or standard error of arbitrary transformations of the MLE. To do this, we need the MLE estimates, and we need the variance-covariance matrix of the MLE

estimates, which we can compute as the inverse of the hessian matrix of the negative-log-likelihood function.

$$VCV_{\theta,MLE} = H_{\theta,MLE}^{-1}$$

Now consider some function of the MLE parameters, $f(\vec{\theta})$. Recalling that the MLE is asymptotically normally distributed such that $\hat{\theta} \sim MVN(\hat{\theta}_{MLE}, VCV_{\theta,MLE})$, then we can compute the approximate variance of our new estimated quantity:

$$f(\theta) \sim Normal(f(\hat{\theta}_{MLE}), \nabla f(\hat{\theta}_{MLE})^T \times VCV_{\theta,MLE} \times \nabla f(\hat{\theta}_{MLE}))$$

That is, the variance of the transformed random variable is:

$$\nabla f(\hat{\theta}_{MLE})^T \times VCV_{\theta,MLE} \times \nabla f(\hat{\theta}_{MLE})$$

The standard error is just the square root of the variance. Finally, you can compute a confidence interval by multiplying the standard error by the desired quantile of the standard normal distribution. For a 95% CI, just multiply the standard error by the upper 0.025 quantile of the z distribution, and add and subtract this quantity from the mean value of the quantity. Here is an example so you can start getting the hang of it!

```
# assume X is a random variable with mean of c(2.2, 4.1)

exp_X = c(2.2,4.1) # expected value of random MVN variable
vcv_X = matrix(c(1,.2,.2,1.5),nrow=2) # vcv matrix of X

# imagine we want to evaluate a transformation of X: 1/(x1^2 + log(x2))

myfun = function(x) 1/(x[1]^2+log(x[2])) # write a function for your transformation

# compute the gradient of the transformation with respect to the components of X,
# evaluated at the mean values of X
f_prime_X = numDeriv::grad(myfun,exp_X)

# apply the delta method to obtain an approximate standard error:
var_fX = (t(f_prime_X) %*% vcv_X %*% f_prime_X)[1,1]

# compute the standard error:
se_fX = sqrt(var_fX)

# compute the critical value for 95% CI
z_crit = qnorm(0.025,lower=F)

# compute confidence interval
ci_fX = myfun(exp_X) + z_crit*se_fX * c(mean=0,lower=-1,upper=1)
ci_fX
```

```
##          mean      lower      upper
## 0.15997474 -0.06366178  0.38361125
```

Note that Bolker's 'emdbook' package has a 'deltavar()' function for computing variances using the delta method. You are welcome to use Bolker's function, but I encourage you to apply the delta method 'from scratch' using the 'numDeriv' package to compute gradients.

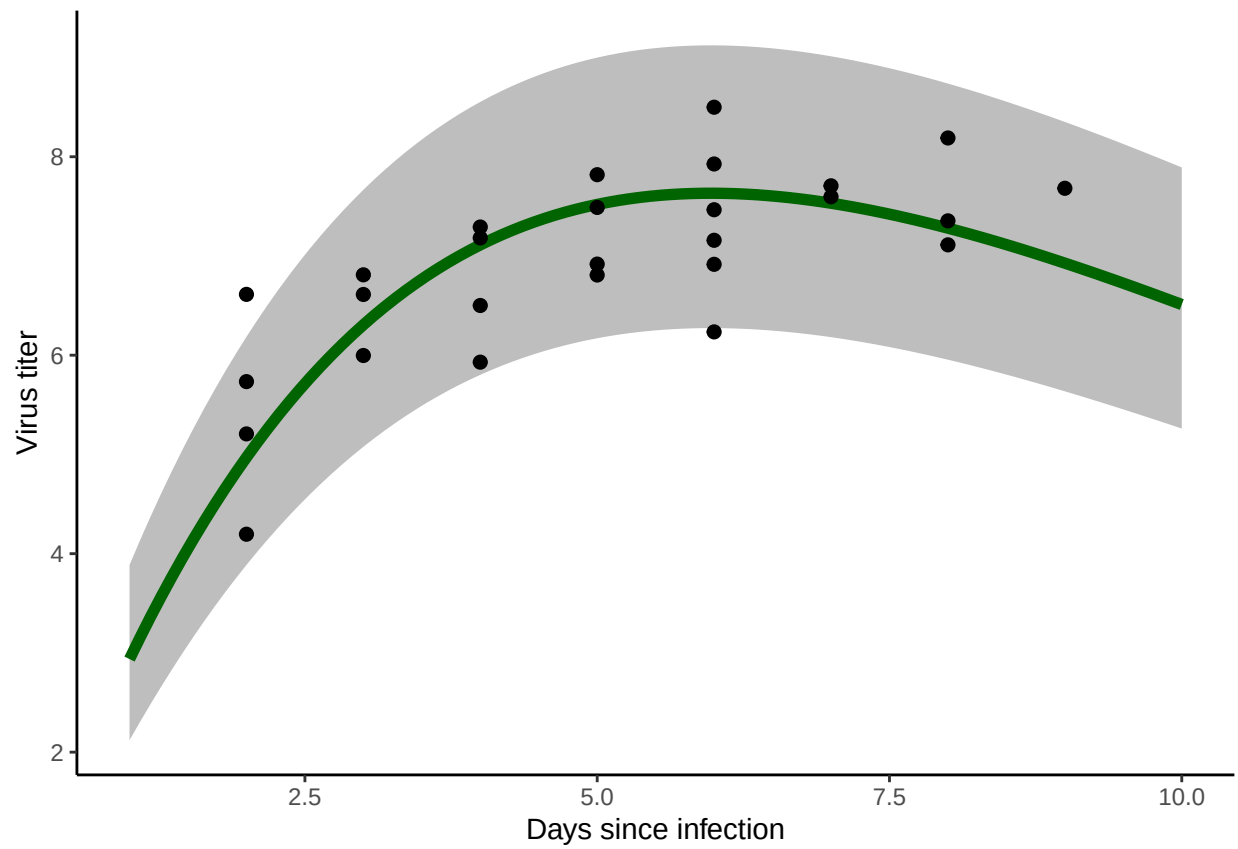
As extra credit, see if you can compute and visualize a 95% prediction interval. You can do this by applying the delta method to compute the 'shape' parameter of the gamma distribution, which is a derived parameter involving all three parameters you estimated in this problem (a, b, and rate).

NOTE: you will need to apply the delta method for every prediction you make- that is, as you vary the number of days since exposure, the variance of the predictions will change.

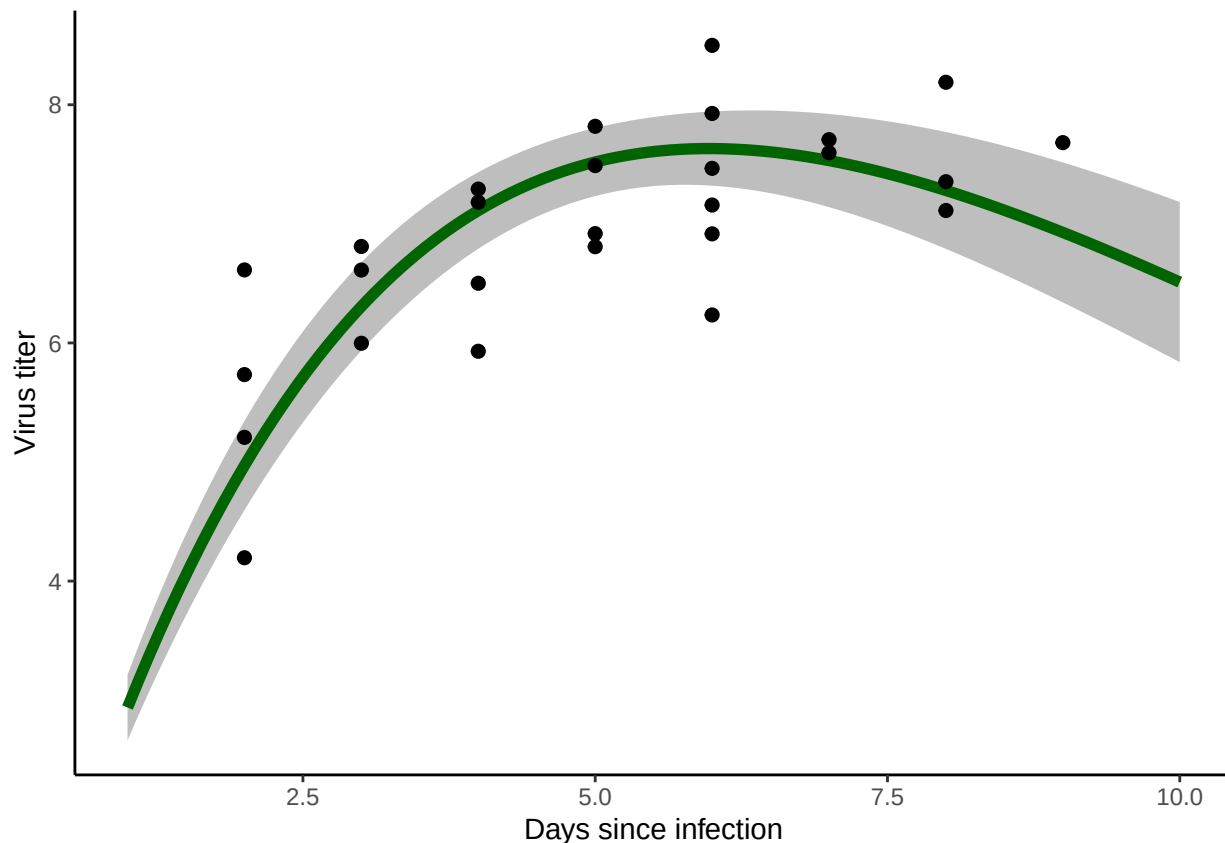
- **input:**
 - mle = named vector containing the maximum likelihood values for the params (length 3: *a* and *b* params from the Ricker model [see below], and the *rate* parameter of the Gamma distribution)(names should be “a”, “b”, and “rate”)
 - H = the hessian matrix obtained from optimizing the negative-log-likelihood function (i.e., from ‘optim()’ with ‘hessian=T’)
 - prediction = a logical T/F indicating whether to generate a prediction interval (if TRUE) or a confidence interval (if FALSE). Set this to FALSE as a default. Since this is an optional problem, you don’t need to incorporate this argument in your algorithm, but please include it anyway!
- **suggested algorithm:**
 - compute variance-covariance matrix (VCM) from the hessian matrix as described above.
 - make a data frame with a column “x” spanning the range of the time-since-infection (predictor variable, in days).
 - add a column with the predictions from the Ricker model based on the MLE parameters (just the Ricker parameters) across the range of times since exposure.
 - use the delta method to compute the standard error for all of your predictions, evaluated at the MLE
 - Finally, visualize prediction intervals around your MLE line.
 - * if prediction=FALSE, then sample many times from a multivariate normal distribution with mean at the MLE and VCM equal to the VCM computed above. For each sampled parameter vector, add the appropriate noise (here, gamma distributed).
 - summarize the upper and lower bound for each
 - compute the margin of error by multiplying the standard error of each parameter by the appropriate quantile of the z distribution (standard normal). In this case you want the 97.5% quantile (make sure you know why!).
- **return:**
 - a matrix: each row is a parameter (in order: a, b, and rate) and there are three columns: first, the mle values, second, the lower bound of the confidence/prediction interval, and third, the upper bound of the confidence/prediction interval.

You can test your function code using something like this:

```
predict_myxRicker2(MLE,H,prediction = T)    # test the function
```



```
predict_myxRicker2(MLE,H,prediction = F)
```



Exercise 4.1c (optional extra credit!): Profile Likelihood confidence interval

In this optional (extra credit) exercise, you will add an option to construct *profile likelihood* confidence intervals instead of using the normal approximation (using the Hessian matrix, above). parameter using repeated calls to the “`optim()`” function.

To do this you will need to modify your likelihood functions and “`optim()`” commands to estimate only the parameters *other than* the one you are trying to get a CI for (because you’ll be fixing that parameter at many values on either side of the MLE).

Again, you can assume that there is a dataframe in your environment named ‘`myxdat`’ with columns named ‘`days`’ and ‘`titer`’ that contain the data.

Since this is for extra credit, I am not giving you a suggested algorithm here. Take this as a fun challenge! You might start by writing down some psuedocode to think through the problem before you start coding. Try to do it without help from a chatbot. If you do use a chatbot, ask it to help find solutions using base R. This course is about building custom algorithms so it’s kind of cheating if you use higher level functions that “hide” the underlying algorithms.

- **input:**

- `mle` = named vector containing the maximum likelihood values for the params (length 3: *a* and *b* params from the Ricker model [see below], and the *rate* parameter of the Gamma distribution)(names should be “`a`”, “`b`”, and “`rate`”)
- `H` = the hessian matrix obtained from optimizing the negative-log-likelihood function (i.e., from ‘`optim()`’ with ‘`hessian=T`’)
- `nll` = the minimum negative-log-likelihood at the MLE
- `profile` = a logical T/F indicating whether or not to use the profile likelihood method. If not,

the function should return confidence intervals using the normal approximation with the hessian matrix.

- **suggested algorithm:**

Up to you!

- **return:**

- a matrix: each row is a parameter (in order: a, b, and rate) and there are three columns: first, the mle values, second, the lower bound of the confidence interval, and third, the upper bound of the confidence interval.

Finally, test the function and compare the profile likelihood method with the hessian method:

```
CI_myxRicker2(MLE,H,MinNLL,profile=T,alpha=0.9)
```

```
##           mle           lb           ub
## [1,]  3.4736659  3.4510419  3.4962900
## [2,]  0.1674329  0.1661746  0.1686911
## [3,] 14.4231248 13.9389819 14.9072678
```

```
CI_myxRicker2(MLE,H,MinNLL,profile=F,alpha=0.9)
```

```
##           mle           lwr           upr
## a      3.4736659  3.4487196  3.4986123
## b      0.1674329  0.1661403  0.1687254
## rate 14.4231248 13.9294345 14.9168152
```

—————End of lab 3—————